

Generative Methods for Motion Planning in Robotics: A Survey

Saad Muhammad Abdul Ghani
Computer Science Department
George Mason University
Fairfax, VA - 22030
sghani2@gmu.edu

Abstract—Generative methods have emerged as a promising approach to improve motion planning in robotics by addressing the limitations of classical planners in high-dimensional and complex environments. This survey provides a comprehensive review of how generative models are employed for robotic motion planning. I categorize these approaches based on their integration into the motion planning pipeline: as standalone generative planners, as hybrid components alongside classical sampling or optimization methods, or as tools for augmenting environment representations. Across these categories, I review each of the 23 papers, summarizing their contributions and highlighting their strengths and weaknesses. Finally, I highlight the open problems that I find to be the most important and suggest potential solutions.

I. INTRODUCTION

Motion planning is a fundamental challenge in robotics, addressing how an agent can navigate its environment while satisfying various constraints to achieve task-specific goals. These constraints may include kinematic and dynamic feasibility, environmental obstacles, and task-related requirements. The nature of planning tasks varies widely across robotic applications; for instance, robotic manipulators must compute valid joint configurations for manipulation, whereas unmanned ground vehicles (UGVs) and aerial vehicles (UAVs) require efficient trajectory generation for autonomous navigation.

Classical motion planners, such as sampling-based and optimization-based methods, offer theoretical guarantees of completeness and asymptotic optimality. However, their efficiency deteriorates in high-dimensional spaces, where feasible motion plans constitute only a small subset of the search space. To address these limitations, learning-based approaches have gained traction, leveraging *Learning from Demonstration (LfD)* and *Reinforcement Learning (RL)* to improve motion planning efficiency and adaptability. Recently, **generative methods**—originally developed for applications in computer vision and natural language processing—have emerged as a compelling alternative for robotic motion planning, either as hybrid enhancements to classical planners or as standalone planning frameworks.

The integration of generative models in motion planning began in the late 2010s, marking a shift from *procedural heuristics* to *data-driven trajectory generation*. Early works explored Conditional Variational Autoencoders (CVAEs) to bias sampling in classical planners, significantly improving planning efficiency over uniform sampling [1]. Over

time, generative models expanded beyond basic navigation, facilitating *robotic manipulation*, *grasping*, and *multi-step trajectory optimization*. A pivotal advancement came with the introduction of Energy-Based Models (EBMs) into cost-function-driven motion optimization, demonstrating the feasibility of incorporating learned priors into optimization-based frameworks [2]. More recent studies have extended generative planning into complex domains such as *joint grasp and motion optimization* [3] and *motion generation conditioned on visual observations* [4], reflecting the growing role of generative methods in high-dimensional, multimodal tasks.

This survey provides a systematic review of generative methods in motion planning, categorizing the various approaches and analyzing their impact on different aspects of planning. The primary contributions of this work are:

- A structured taxonomy of generative methodologies in motion planning, categorizing approaches based on their role in the planning pipeline and their integration with classical methods.
- A critical analysis of the strengths, limitations, and emerging trends across different generative frameworks.
- An overview of open research problems and potential future directions for generative motion planning.

The remainder of this paper is structured as follows:

- **Section II** introduces fundamental concepts in motion planning and generative modeling techniques.
- **Section III** presents the proposed taxonomy, organizing surveyed approaches into a structured framework.
- **Section IV** discusses the surveyed papers contributions in each category, highlighting their advantages and challenges.
- **Section V** identifies open challenges and future research opportunities.
- **Section VI** provides concluding insights.

This survey complements existing works, such as recent surveys on *generative learning in Learning from Demonstration (LfD)* [5] and the broader application of *machine learning in motion planning* [6]. Unlike [6], which primarily focuses on navigation, this survey extends its scope to include manipulation and grasping. Additionally, unlike [5], which focuses exclusively on generative models, this survey also considers discriminative models and procedural methods used in a generative capacity, offering a broader perspective

on motion planning techniques.

II. PRELIMINARIES

This section introduces important concepts relevant to this survey. It introduces the motion planning pipeline, formulates the motion planning problem, details the different generative models surveyed and finally details the generative methods (discriminative models and procedural methods) used.

A. Motion Planning Pipeline

Motion planning consists of generating a sequence of actions $P^* = \{u_0, u_1, \dots\}$ that move a robot from its starting state s to a goal state g in a given environment E :

$$P^* = \operatorname{argmin}_{P \in \mathcal{P}} J(E, s, g), \quad (1)$$

where $J(\cdot)$ is a cost function that contains constraints such as kinematic, dynamic, and environmental obstacles, and $\mathcal{P}$ is the robot's plan space. Generally, most motion planning tasks require a high-level course plan as well as a low-level fine-grained plan. This course plan may be multi-objective and ultimately moves the robot from s to g but the intermediate steps in between are handled by the fine-grained plan that plans around the immediate environment and handles all kino-dynamic and environmental constraints. As such, most classical systems decompose J and tackle it in a hierarchical manner using a global planning subsystem and a local planning subsystem. These subsystems are further divided into a *perception module* and a *planner*. The perception module is responsible for creating a representation of the environment while the planner creates a motion plan on this representation.

However, the works surveyed in this paper deviate from this traditional hierarchical decomposition. Instead, they either employ *end-to-end generative methods* or focus exclusively on the local planning subsystem. Based on how generative methods are integrated into the motion planning pipeline, we categorize the surveyed papers into three primary groups that form our primary taxonomy:

- 1) Using generative methods purely as planners IV-A.
- 2) Using generative methods with classical planners as hybrid planners IV-B.
- 3) Using generative methods augment representation of environment IV-C.

B. Motion Planning Problem Definition

Robot motion planning in obstacle-filled environments relies on configuration space, Q , which is divided into $Q = Q_{free} \cup Q_{obst}$, where Q_{free} consists of valid configurations and Q_{obst} contains those obstructed by obstacles or constraints. As described in Eq. (1), motion planning involves generating an optimal sequence of actions P^* to move the robot from its start s to its (local or global) goal g , while minimizing the cost function $J(E, s, g)$. In this context, an optimal motion plan $p^* \in \mathcal{P}$ is a sequence $p^* = \{u_t\}_{t=0}^{H-1}$ with actions $u_t \in \mathcal{U}$ from the robot's action space over time horizon H . The motion planning function $f(\cdot)$ generates a optimal plan $p^* = f(Q_{free}|s, g)$, ensuring

all intermediate configurations satisfy $c_t \in Q_{free} \forall t$ and that $p^* = \operatorname{argmin}_{p \in \mathcal{P}} J(Q_{free}, s, g)$.

Classically, motion planners are categorized either as sampling-based motion planners (SBMP) or optimization-based motion planners (OBMP). Sampling-based planners [7], [8] operate by implicitly representing Q_{free} by a set of probing samples and making connections to neighboring samples. As more samples are added, the implicit representation is able to model Q_{free} completely and allows for asymptotic guarantees on the completeness (a feasible plan will be found) and optimality (the plan converges to the optimal plan) of p^* . In contrast, optimization-based planners [9], [10] iteratively refine an initial plan into an optimized solution through methods such as gradient descent or stochastic optimization. The cost function $J(\cdot)$ consists of different costs such as c_g , the distance to the goal g , c_{obst} , the cost of colliding with Q_{obst} , c_{sm} , the cost for non-smooth plans, and potentially other costs. The motion planning function $f(\cdot)$ then aims to minimize the weighted sum of all costs and can be succinctly written as $p^* = \operatorname{argmin}_{p \in \mathcal{P}} \sum_i \lambda_i c_i(p, s, g)$, where λ_i is the weight associated with cost term c_i .

As mentioned before, the effectiveness of these classical approaches diminish significantly in complex, high-dimensional scenarios where feasible motion plans occupy only a small subset of the entire state space. Sampling-based methods, which typically sample uniformly, can only address this issue through exhaustive computational efforts, while optimization-based methods are susceptible to local optima and are heavily influenced by their initialization.

Given these challenges, **hybrid approaches** integrating generative models with classical planners leverage either SBMPs or OBMPs to enhance efficiency and adaptability. The taxonomy in Fig. 1 distinguishes between these hybrid approaches, highlighting the inherent differences in integrating generative models with sampling-based and optimization-based planners.

C. Generative Models

Generative models are powerful machine learning frameworks capable of creating novel data by modeling complex probability distributions. Formally, given an unknown target vector $\mathbf{y}$ and a known input vector $\mathbf{x}$, generative models focus on modeling the joint distribution $p(\mathbf{y}, \mathbf{x})$. These models perform inference by applying the product rule and computing the conditional probability:

$$p(\mathbf{y}, \mathbf{x}) = p(\mathbf{y}|\mathbf{x}) \cdot p(\mathbf{x}).$$

In contrast, discriminative models, such as those employed for image classification, directly model the conditional distribution $p(\mathbf{y}|\mathbf{x})$. While discriminative models primarily differentiate labels y given input data $\mathbf{x}$, generative models explicitly characterize the distribution of $\mathbf{x}$, allowing them to sample novel data points from $p(\mathbf{x})$.

Additionally, conditional generative models explicitly model the distribution $p(\mathbf{x}|y)$, enabling the generation of novel data conditioned on specific labels y . This conditional relationship can be derived using Bayes' theorem, treating

$p(y)$ as a normalizing constant. In either case, the root of the problem lies in modeling $p(\mathbf{x})$ which is nontrivial.

This survey explores four primary categories of generative models, each employing distinct methods to approximate $p(\mathbf{x})$. Following [5], these categories include:

Autoregressive models model $p(\mathbf{x})$ sequentially using the product rule of probability:

$$p(\mathbf{x}) = p(x_0) \prod_{i=1}^D p(x_i | \mathbf{x}_{0:i}),$$

where $\mathbf{x} \in \mathbb{R}^D$ and $\mathbf{x}_{0:i}$ denotes the sequence $\{x_0, x_1, \dots, x_{i-1}\}$. Training ARMs involve maximizing the likelihood of the training data under the model.

Sampling models capture the distribution of $\mathbf{x}$ conditioned on a noisy sample. Formally,

$$\mathbf{z} \sim p(\mathbf{z}), \quad \mathbf{x} \sim p(\mathbf{x}|\mathbf{z}),$$

where $\mathbf{z} \in \mathbb{R}^z$ and $p(\mathbf{z})$ is typically sampled from an easily manageable distribution, such as the standard normal $\mathcal{N}(0, \mathbf{I})$. Different classes of sampling models approach the problem in unique ways. The following three types are surveyed in the paper:

- **Variational Autoencoders (VAEs)** [11] model $p(\mathbf{x}|\mathbf{z})$ using an encoder-decoder architecture. The encoder learns a posterior distribution of the latent variable $q_\phi(\mathbf{z}|\mathbf{x})$, and the decoder learns to model the likelihood, $p_\theta(\mathbf{x}|\mathbf{z})$. Training involves maximizing the Evidence Lower Bound (ELBO):

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})] - KL(q_\phi(\mathbf{z}|\mathbf{x}) || p(\mathbf{z})),$$

where KL is the Kullback–Leibler divergence.

- **Generative Adversarial Networks (GANs)** [12] use two networks, a generator G and a discriminator D . The generator models $p(\mathbf{x}|\mathbf{z})$ by mapping noisy samples from a latent space $p(\mathbf{z})$ to data space. The discriminator distinguishes between real data $\mathbf{x}$ and generated data $G(\mathbf{z})$. Training the generator and discriminator is done using a minimization-maximization optimization problem:

$$\min_G \max_D \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

- **Normalizing Flows (NFlows)** [13] model $p(\mathbf{x})$ using a sequence of invertible, differentiable transformations $f_\theta = f_1 \circ f_2 \circ \dots \circ f_K$ applied to a simple latent distribution $p(\mathbf{z})$. Since f is invertible, $p(\mathbf{x})$ can be computed exactly by the change of variables formula

$$p(\mathbf{x}) = p_{\mathbf{z}}(f_\theta^{-1}(\mathbf{x})) \left| \det \frac{\partial f_\theta^{-1}}{\partial \mathbf{x}} \right|.$$

The parameters θ are trained by maximizing the log-likelihood of the observed data.

Energy Based Models (EBMs) [14] produce a scalar value given an input vector using an *energy function* $E(\mathbf{x})$. The distribution of $\mathbf{x}$ is a Boltzmann distribution:

$$p(\mathbf{x}) \propto \exp(-E(\mathbf{x})).$$

Due to their implicit form, sampling from an EBM requires Markov Chain Monte Carlo (MCMC) methods, Langevin Dynamics being the most popular option. Training an EBM requires estimating the gradient of the energy function, often done via contrastive divergence (CD) [15]. In CD, a set of samples is drawn using a few steps of MCMC sampling from the model distribution and the gradient of the log-likelihood is approximated by comparing the model's energy function on these *negative* generated samples to that of the *positive* training data. Mathematically, contrastive divergence updates the model parameters by approximating the gradient of the log-likelihood as:

$$\nabla_\theta \log p_\theta(\mathbf{x}) \approx -\mathbb{E}_{p_{\text{data}}(\mathbf{x})} [\nabla_\theta E_\theta(\mathbf{x})] + \mathbb{E}_{p_{\text{model}}(\mathbf{x})} [\nabla_\theta E_\theta(\mathbf{x})].$$

Diffusion models [16] are similar to sampling models in $\mathbf{x}$ is generated from a noisy sample. However, diffusion models generate data through an iterative denoising process $p_\theta(\mathbf{a}_{k-1}|\mathbf{a}_k)$ that gradually removes noise from a sample of a standard normal distribution, $\mathbf{a}_N \sim \mathcal{N}(0, \mathbf{I})$. This denoising process is the inverse of a forward diffusion process $q(\mathbf{a}_{k+1}|\mathbf{a}_k)$ that gradually adds noise to a data point, $\mathbf{a}_0 = \mathbf{x}$. Formally,

$$p(\mathbf{a}_0) = \int p(\mathbf{a}_N) \prod_{k=1}^N p_\theta(\mathbf{a}_{k-1}|\mathbf{a}_k) d\mathbf{a}_{1:N}.$$

The parameters θ can be optimized by minimizing a variational bound on the negative log-likelihood of the reverse process, $\theta^* = \text{argmin}_\theta - \mathbb{E}[\log p_\theta(\mathbf{a}_0)]$.

This survey will highlight how the aforementioned generative models are being used in varying applications and discuss how they are being implemented with respect to the motion planning pipeline (II-A).

D. Generative Methods

The methods discussed in this section differ from the generative models in II-C as they do not explicitly model the underlying data distribution $p(\mathbf{x})$. However, they are still used to generate data that aids motion planning and thus fall within the scope of this survey. We categorize these methods into two main types:

Discriminative Models [17]–[19] model the conditional probability distribution $p(\mathbf{y}|\mathbf{x})$, distinguishing them from traditional generative models that learn the full data distribution $p(\mathbf{x})$. These models can be leveraged for data generation, the following two ways are explored:

- 1) **Density Estimation** [19]: A neural network is trained to estimate the parameters of a probability distribution, commonly a Gaussian:

$$p(\mathbf{y}|\mathbf{x}) = \mathcal{N}(\mathbf{y}|\mu(\mathbf{x}), \Sigma(\mathbf{x})), \quad (2)$$

where $\mu(\mathbf{x})$ and $\Sigma(\mathbf{x})$ are neural network outputs representing the mean and covariance matrix, respectively.

- 2) **Mixture Density Models (MDNs)** [17]: To capture multi-modal distributions, a Mixture of Gaussians model is employed:

$$p(\mathbf{y}|\mathbf{x}) = \sum_{i=1}^K \pi_i(\mathbf{x}) \mathcal{N}(\mathbf{y}|\mu_i(\mathbf{x}), \Sigma_i(\mathbf{x})), \quad (3)$$

where K is the number of mixture components, $\pi_i(\mathbf{x})$ are the mixing coefficients satisfying $\sum_i \pi_i = 1$, and each component is parameterized by its own mean $\mu_i(\mathbf{x})$ and covariance $\Sigma_i(\mathbf{x})$. This formulation allows the model to learn multi-modal trajectory distributions, enabling better uncertainty modeling in motion planning.

Additionally, *discriminant functions* learn a direct mapping $f : \mathbf{x} \rightarrow \mathbf{y}$, where a function approximator (e.g., a neural network) maps input features to outputs without explicitly modeling uncertainty. Novel data can then be generated by inputting unseen $\mathbf{x}$ values into $f(\mathbf{x})$ [18].

Procedural Methods take an algorithmic approach to data generation, commonly used for augmenting environment representations in motion planning. These methods rely on predefined rules rather than learned distributions and have been widely used in simulation-based learning [20]–[22].

Similar to II-C, this survey will explore how these generative methods are integrated into motion planning, particularly their role within the motion planning pipeline II-A.

III. AREA TAXONOMY

In this section, I discuss the taxonomy presented in Fig. 1 and its associated legend (Fig. 2). The taxonomy organizes the surveyed papers along three key dimensions. For each dimension, I briefly outline the role of generative methods before analyzing the joint taxonomy in greater detail.

The **first and primary categorization** is based on how generative methods are integrated into the motion planning pipeline. Across the 23 papers surveyed, generative methods are used in three distinct ways: (1) as a *pure generative planner*, (2) as a *hybrid planner* in combination with a classical motion planning method, or (3) to *augment environment representation* by enhancing the perception module. Within the hybrid category, generative methods are further divided into those used with *sampling-based planners* and those with *optimization-based planners*. The lower number of papers in the latter category highlights the difficulty of integrating generative models into optimization-based frameworks, which rely heavily on cost functions. Directly embedding generative methods into cost function formulations is a non-trivial task, and only one work [2] successfully achieved this by integrating an Energy-Based Model (EBM) into a trajectory optimization framework. In contrast, the remaining seven hybrid approaches use generative models to construct a posterior distribution that classical motion planners can efficiently sample from, making integration with sampling-based methods more straightforward.

The **second categorization** distinguishes application domains based on increasing degrees of freedom (DoF). A notable trend is the relatively low number of papers applying generative methods to *robotic manipulation and grasping*. Unlike general motion planning, grasping requires direct interaction with objects rather than obstacle avoidance, making it inherently more complex. This gap suggests that while generative models have been widely explored in motion

planning, their use in manipulation tasks—especially for grasping—remains an area for further research.

The **third categorization**, which can be inferred from Fig. 2, reveals a clear preference for *sampling-based generative models* due to their flexibility in leveraging latent space representations. In contrast, *Energy-Based Models (EBMs)* are less commonly used, likely due to their implicit nature and computational challenges. However, the rapid emergence of *diffusion models*, despite being relatively new [16], is evident in the number of recent works adopting them. This reflects the increasing popularity of diffusion models for motion planning tasks, suggesting their strong representational power and adaptability.

When analyzing all three dimensions together, several key trends emerge. First, **sampling-based generative models remain highly effective and widely applicable**. For instance, the relatively simple *Conditional Variational Autoencoder (CVAE)* has been successfully applied across varying degrees of freedom [1], demonstrating its versatility. Second, there is a **noticeable gap in applying generative methods for hybrid manipulation and grasping tasks**. Interestingly, two of the three papers in this domain [4], [22] employed a *combination of generative methods*—one enhancing state representation and the other learning the downstream task—indicating that integrating multiple generative models can be particularly powerful. Both papers also utilized *diffusion models*, underscoring their strong representational and learning capabilities (though Chi et al. (2024) is categorized as an Autoregressive Model (ARM), it employs a Time-Series Diffusion Transformer for planning).

A third trend is observed in the **shift to richer, more complex models**. This can be seen in the progression of the Learning from Hallucination (LfH) methods [19]–[21]. The earlier two works relied solely on *procedural methods* and were constrained to planar robots, whereas the latest method adopted a *discriminative model*, enabling its application to aerial vehicles (UAVs). This shift highlights the advantages of moving toward richer and more sophisticated generative approaches. A similar pattern is seen in the evolution of diffusion models in challenging motion planning domains. Following the seminal work on *integrating an EBM into cost function optimization* [2], diffusion models proliferated in complex domains [3], [4], [23], [24]. This trend reflects the growing role of *diffusion models as priors in optimization-based motion planning*, reinforcing their value in tackling high-dimensional, difficult planning problems.

In summary, the taxonomy reveals that **sampling-based methods remain dominant due to their flexibility, diffusion models are rapidly gaining traction, and hybrid approaches for manipulation and grasping remain under-explored**. The trends suggest a shift toward *more expressive and adaptable generative models*, with increasing efforts to integrate them into both sampling-based and optimization-based planning frameworks.

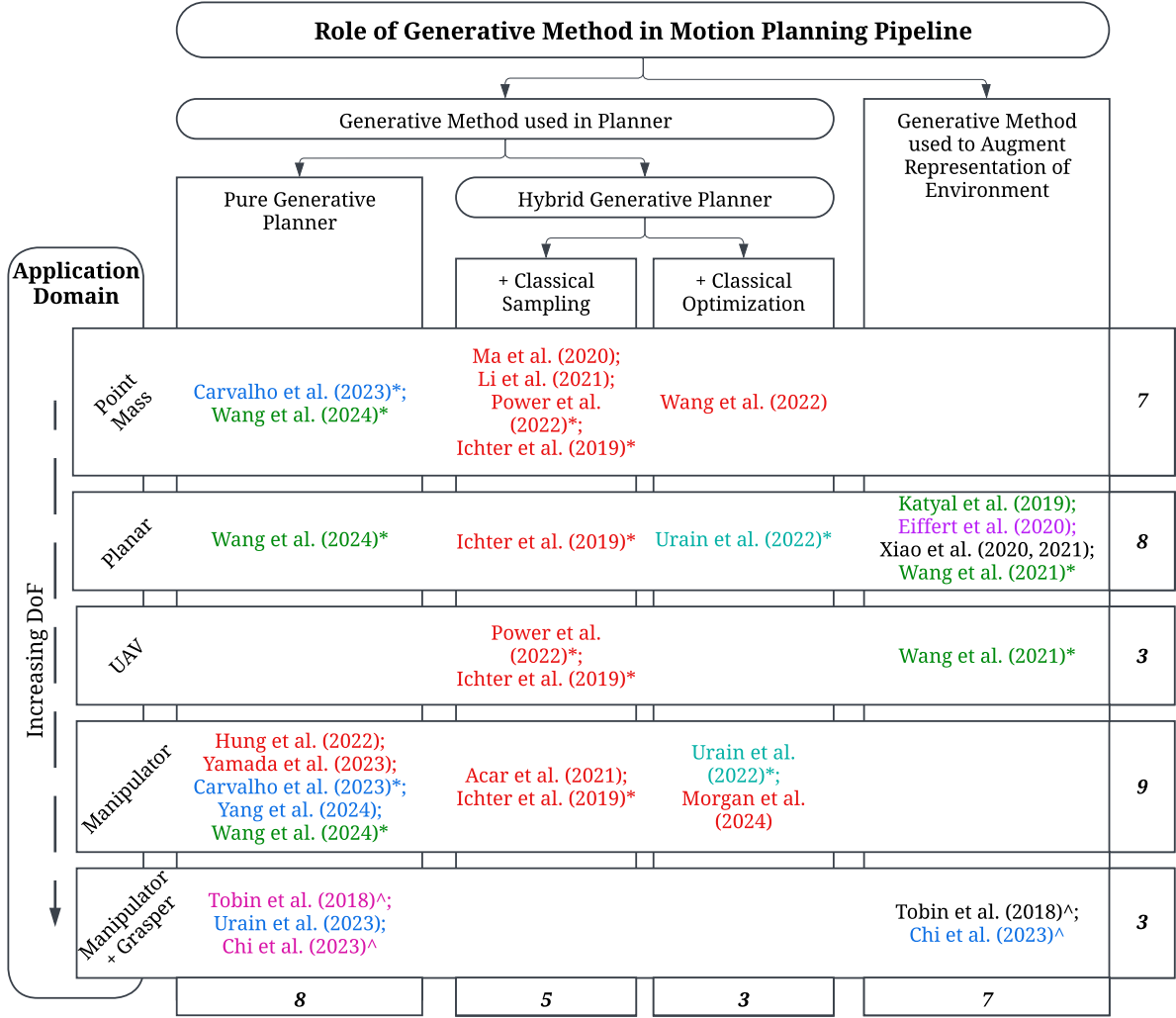


Fig. 1: The complete taxonomy of generative methods in motion planning. The taxonomy is organized along three dimensions: **vertically**, by the role of the generative method in the navigation pipeline; **horizontally**, by the application domain; and **by color**, to indicate the type of generative method used (see the legend in Fig. 2). The numbers at the right and bottom indicate the total unique papers in each application domain and role, respectively. Papers marked with * appear in multiple application domains, while those marked with ^ are listed under multiple roles, as they employ different generative methods for different stages of the navigation pipeline.

IV. TAXONOMY-BASED SURVEY

This section will discuss each surveyed paper in the context of their role in the motion planning pipeline.

A. Generative Methods as Planners

Generative models offer a data-driven alternative to traditional motion planners by learning a distribution of feasible trajectories from experience. Rather than searching from scratch, they generate entire paths in one shot, avoiding the step-by-step decisions of classical planners [24]. Using neural generative models like VAEs or diffusion models, they efficiently produce smooth, collision-free trajectories while reducing computation time [25]. This approach also enables the integration of complex constraints and high-dimensional sensor inputs, enhancing adaptability in real-world scenarios.

Reaching Through Latent Space (LSPP) [26] introduces

planning in the latent space of a Variational Autoencoder (VAE) [11], encoding valid robot configurations for efficient trajectory optimization. A learned collision classifier enables constraint checking, allowing gradient-based obstacle avoidance. LSPP achieves success rates comparable to classical planners while significantly reducing planning time by leveraging learned priors. However, its reliance on predefined obstacle shapes limits generalization to complex, unmodeled environments, restricting applicability in real-world scenarios.

Activation Maximisation Planning in Latent Space (AMP-LS) [25] improves LSPP by incorporating learned scene embeddings, using SceneCollisionNet to model obstacles from raw point cloud data. This enables *zero-shot generalization* to novel environments and dynamic scenar-

Generative Method	# Papers
Autoregressive Models	3
Sampling Based Models	9
Entropy Based Models	1
Diffusion Models	4
Discriminative Models (Generative Methods)	3
Procedural Methods (Generative Methods)	3

Fig. 2: The color legend for Fig. 1, showing the different generative methods and the total number of papers associated with each method.

ios. A VAE encodes robot joint configurations, while a collision-check loss refines latent-space optimization, ensuring collision-free planning. AMP-LS achieves success rates comparable to LSPP and classical planners but is an order of magnitude faster, making real-time reactive planning feasible. However, its reliance on learned embeddings means performance depends on training data coverage and accuracy.

Concurrently, another family of works [3], [4], [23], [24] uses diffusion models to generate motion plans. **Motion Planning Diffusion (MPD)** [23] treats motion planning as a generative sampling problem, learning a diffusion-based prior over trajectories from successful plans. At runtime, it samples feasible paths via a denoising process, capturing multi-modal distributions and adapting to unseen obstacles. This approach avoids local minima and inefficiencies of random sampling, leading to smoother, collision-free motions. However, its performance depends on the quality of training data, and inference can be slower due to iterative denoising. Despite this, MPD demonstrates the potential of diffusion models for robust, data-driven motion planning.

SE(3)-DiffusionFields [3] unifies grasp selection and motion planning by learning a generative cost field over SE(3) using a diffusion model. This smooth, differentiable cost field guides gradient-based optimization, ensuring feasible, collision-free trajectories while avoiding suboptimal grasp choices. Unlike traditional methods that decouple grasping and motion, this approach improves performance in complex pick-and-place tasks. Its ability to model multi-modal grasp preferences enhances flexibility, but reliance on accurate training data may limit generalization to novel scenarios. Despite this, SE(3)-DiffusionFields demonstrates the advantages of diffusion models in integrated manipulation planning.

Planning-as-Inpainting [24] frames trajectory generation as an image inpainting problem, where a diffusion model fills in a missing path on an occupancy map or visual observation. This approach leverages high-dimensional sensory data, ensuring obstacle-aware, coherent paths. Unlike incremental planners, it generates complete trajectories in one pass, reducing shortsighted decisions. Additionally, it

integrates language instructions to handle partial observability, improving goal alignment. However, reliance on accurate spatial context means errors in sensory input can degrade performance. Despite this, Planning-as-Inpainting demonstrates the potential of diffusion models for complex, real-world scenario planning.

Diffusion Policy [4] applies diffusion models to visuomotor policy learning, treating decision-making as a generative trajectory sampling process. A conditional diffusion model iteratively refines noisy action sequences into goal-directed behavior, enabling multi-modal action distributions and scaling to high-dimensional spaces. Compared to adversarial and autoregressive models, it offers greater training stability through likelihood optimization. Empirically, it achieves state-of-the-art performance, improving success rates by 46.9% over prior imitation learning methods across 15 tasks. Features like receding-horizon planning and a time-series diffusion transformer enhance long-horizon execution, enabling real-world deployment. Diffusion Policy demonstrates how generative models can effectively capture complex visuomotor behaviors for end-to-end robot control.

Beyond generative models, **Learning-Based Motion Planning with Mixture Density Networks (MNP)** [17] predicts a distribution over possible paths rather than committing to a single solution. Many planning problems are inherently multimodal, but conventional planners often collapse paths into a suboptimal average. MNP models multimodality by learning multiple Gaussian components from expert demonstrations, generating diverse trajectory candidates. Given a point cloud and start-goal query, MNP outputs multiple feasible routes, improving success rates in complex scenarios. It achieves real-time planning with a single forward pass and outperforms deterministic planners like MPNet and sampling-based methods like Informed RRT* and BIT*. By combining fast inference with multimodal learning, MNP effectively handles environments with multiple navigation options.

Overall, VAE-based, diffusion-based, and mixture-based planners each offer trade-offs in performance, generalization, and efficiency. VAE-based methods (LSPP, AMP-LS) excel in computational speed by leveraging latent-space gradients, with AMP-LS planning an order of magnitude faster than standard samplers. However, they typically generate a single solution per query, requiring re-optimization for alternative paths. Diffusion-based methods invest more computation in iterative sampling but capture rich, multimodal trajectory distributions, enhancing generalization to novel or high-dimensional scenarios. They handle ambiguity well but are slower due to multi-step denoising. Mixture density networks (MNP) balance these approaches, learning multiple trajectory modes with minimal computational overhead, improving success in multi-modal planning tasks. In summary, VAEs prioritize speed, diffusion models offer expressive trajectory distributions, and mixture-based planners provide diversity with fast inference.

B. Generative Methods as Hybrid Planners

Generative models enhance classical motion planners by improving sampling efficiency and planning success. These methods use learned priors to guide planners toward promising regions, combining data-driven insights with classical guarantees. This section reviews these methods, analyzing their strengths, weaknesses, and the trade-offs between sampling and optimization-based integration.

Learning Sampling Distributions for Robot Motion Planning [1] propose a framework to bias sampling-based planners using a conditional VAE. The CVAE is trained on example solution paths, conditioned on the start, goal, and environment, enabling it to generate samples that guide planners like RRT and PRM toward feasible paths. This improves efficiency, yielding significantly higher success rates and faster convergence compared to uniform sampling. However, generalization remains a challenge, as the model may struggle in environments that differ from training data, requiring iterative retraining. Despite this, the approach effectively combines the exploration capability of learning-based methods with the completeness guarantees of classical planners.

Approximating Constraint Manifolds using Generative Models [27] tackles constrained motion planning, where valid configurations are scarce. The authors train a Conditional VAE (CVAE) and a Conditional GAN (CGAN) to learn constraint manifolds and generate valid samples, reducing the need for rejection sampling. This approach is planner-independent and accelerates sampling-based methods like RRT and PRM while maintaining a 100% success rate. However, CGANs may suffer from mode collapse, and CVAEs can produce out-of-distribution samples, requiring additional filtering. Despite these challenges, the method significantly improves planning efficiency with a one-time training cost.

Conditional GAN for Optimal Path Planning (CGAN-RRT*) [28] enhances RRT* by training a Conditional GAN (CGAN) on example paths to generate a probability distribution over feasible routes. Instead of uniform sampling, RRT* leverages this learned distribution, focusing growth on optimal paths and significantly accelerating convergence. Compared to standard RRT* and Informed-RRT*, CGAN-RRT* finds shorter paths faster. The method generalizes well across different map types but requires a large training set, and its performance may degrade in novel environments. Despite this, it demonstrates how GANs can effectively guide sampling-based planners for more efficient path planning.

GAN-Accelerated Particle Swarm Optimization for Path Planning (LPSO) [29] combines GANs with Particle Swarm Optimization (PSO) to accelerate convergence. A generative double-adversarial network pre-processes the map to identify feasible regions, which guide the swarm towards optimal paths while ensuring obstacle avoidance. This improves efficiency over traditional PSO and ACO, reducing unnecessary exploration and adapting well to both simple and complex environments. However, performance depends on the GAN's accuracy—misidentified feasible regions can

lead to suboptimal paths. Training is also complex, but once completed, the two-stage process runs efficiently, achieving faster path planning without sacrificing solution quality.

Recurrent Generative Model for Heuristic Path Planning (RGM) [30] proposes a GAN-based RGM to generate heuristics for path planning, guiding algorithms like RRT*. The model refines its outputs recurrently, improving path feasibility by enforcing connectivity and safety through two discriminators. This produces a heuristic cost map, allowing RRT* to explore more efficiently and reduce unnecessary node expansions. Compared to single-pass generative models, RGM improves accuracy and generalization, adapting to unseen environments. However, it requires significant training time and tuning, and scaling to higher-dimensional spaces may increase computational demands. Despite this, it effectively integrates learned heuristics into classical planners, accelerating search while maintaining solution quality.

Normalizing Flows for MPC (FlowMPPI) [31] integrates a conditional Normalizing Flow into Model Predictive Control (MPC) to improve sampling efficiency. The model learns a distribution of optimal trajectories based on the robot's state, goal, and environment, allowing MPC to sample goal-directed, collision-free paths instead of random control sequences. An out-of-distribution (OOD) projection step enhances generalization to novel environments. FlowMPPI outperforms standard MPC baselines, offering real-time performance and improved robustness. However, training is data-intensive, and retraining may be required if dynamics or tasks change. Despite this, it effectively combines learning-based trajectory generation with the reliability of MPC for dynamic motion planning.

CppFlow: Generative IK for Cartesian Paths [32] combines a learned generative inverse kinematics solver with classical optimization for Cartesian path planning. A conditional Normalizing Flow (IKFlow) generates self-collision-free joint-space trajectories, achieving up to 129× faster planning than traditional methods. To ensure precision, CppFlow refines these trajectories using differentiable optimization and search. It significantly improves success rates, even in challenging cases where classical planners fail. However, IKFlow does not consider external obstacles, requiring optimization for final adjustments. Despite this, CppFlow effectively merges generative models and classical planning, balancing speed and accuracy.

EBM Priors for Trajectory Optimization [2] integrates Energy-Based Models (EBMs) into trajectory optimization to mitigate local minima in motion planning. EBMs, trained on expert demonstrations, serve as learned priors, guiding optimization towards feasible regions. By incorporating smoothness regularization and occasional resampling, the approach improves convergence and robustness across multi-modal solution spaces. Empirical results show superior success rates over baseline optimizers, with effective real-world transfer. However, EBMs require complex training, including contrastive divergence, and their evaluation adds computational overhead. Unlike other generative models, EBMs lack direct sampling, relying on MCMC or integration

with an optimizer to generate trajectories.

Different generative models offer distinct advantages in motion planning. VAEs provide efficient sampling and broad coverage, making them useful for generalization, though their samples may require filtering for accuracy. GANs generate highly realistic, constraint-satisfying paths, focusing planning in promising regions but risk mode collapse, requiring careful training [27]. Normalizing Flows balance VAEs and GANs by capturing the full distribution with efficient, exact sampling, making them ideal for probabilistic trajectory planning, though their form restricts adaptability to any domain. Energy-Based Models (EBMs) do not generate samples directly but guide optimization with an energy landscape, effectively biasing search towards feasible solutions. VAEs and Flows excel in coverage and generalization, while GANs and EBMs prioritize accuracy and feasibility. The best choice depends on the need for broad exploration (VAEs, Flows) or precise, high-quality solutions (GANs, EBMs).

Hybrid motion planners integrate generative models with either sampling-based or optimization-based methods, each with trade-offs. Sampling-based planners gain efficiency while maintaining robustness and probabilistic guarantees, allowing them to recover from imperfect generative suggestions. They are ideal for high-dimensional geometric planning but may still explore randomly if the learned bias is imperfect. Optimization-based planners leverage generative priors for rapid, smooth trajectory generation, excelling in dynamic tasks like navigation and manipulation. However, they lack global exploration and can fail if the learned prior is inaccurate. Sampling-based hybrids are more robust and generalizable, while optimization-based hybrids are faster and more precise but rely on accurate training data. Some methods combine both approaches, using generative models for broad guidance and local refinement, balancing efficiency and adaptability.

C. Generative Methods to Augment Representation of Environment

Generative techniques enhance a robot’s understanding beyond direct observations. This section covers three key applications: generating data from past navigation, extending maps beyond sensor range, and modeling social interactions. These methods provide synthetic yet plausible experiences, improving decision-making.

One prominent approach to safe data generation is the **Learning from Hallucination (LfH)** approach [19]–[21] which offers an alternative to collecting supervised data for Imitation Learning. In LfH, a robot first explores in a completely obstacle-free environment and collects experiences, which are then augmented by ‘hallucinating’ (different from hallucinations in Large-Language Models) imaginary obstacle configurations into those safe trajectories. This yields training examples for navigating highly constrained spaces without ever risking a collision during data collection. However, the initial LfH hallucinates the most constrained obstacle configuration leading to overly conservative planning and a reliance on runtime hallucinations. A subse-

quent improvement proposed **Hallucinated Learning with Sober Deployment (HLSD)** [21], which eliminates the need for any runtime hallucinations. Additionally, HLSD hallucinated minimal obstacle configurations that allowed for less conservative planning leading to a performance boost. Another extension called **Learning from Learned Hallucination (LflH)** [19] replaced procedurally generated obstacle configurations with a learned generative method, specifically an encoder-decoder in which the encoder is a density estimator. Instead of designing how to add virtual obstacles, LflH learns to generate obstacle configurations, in a self-supervised fashion, for which the robot’s recorded open-space trajectories would have been optimal. This removes the need to tailor hallucination logic to specific robot types or assumptions (e.g. fixed planning horizons), making the approach more general; experiments with both ground and aerial robots showed LflH can exceed prior LfH methods’ performance. In summary, hallucination-based methods produce synthetic obstacles or entire scenarios from past data to enrich the robot’s state space during learning. Their strengths lie in safe data generation and improved performance in constrained environments, while their weaknesses (progressively addressed by each variant) include the need for careful scenario design or realism gaps that can cause overly cautious behavior if not mitigated.

This work employs **domain randomization** [22] to generate diverse synthetic training data for robotic grasping by procedurally varying object meshes and properties. This exposure enables the model to generalize to real-world objects without direct training. A key strength is its ability to overcome data scarcity and prevent overfitting by training on a wide range of object geometries. However, a weakness is that synthetic objects may lack fine-grained details or material properties, leading to potential failures on unseen real-world objects. Despite this, the approach achieves 92% real-world grasp success, demonstrating domain randomization’s effectiveness in scaling deep learning for robotic grasping.

Generative methods can extend a robot’s state representation by predicting unseen parts of the map. **Generative Uncertainty-Aware Occupancy Map Prediction** [18] uses a U-Net autoencoder to infer obstacles and free space beyond the robot’s current sensor range. By generating multiple hypotheses, it estimates uncertainty, guiding exploration toward areas with high variance. This approach enhances planning by anticipating unseen obstacles and reducing redundant exploration. However, its accuracy depends on training data quality—if the real environment differs significantly, predictions and uncertainty estimates may be unreliable, potentially misleading the planner.

Generative models have been applied to *social interaction* modeling in robot navigation, predicting how humans, vehicles, or other agents will respond to the robot’s movements. **Planning using Generative RNNs and MCTS** [33] improves upon standard motion models by using a generative Recurrent Neural Network (RNN) to simulate realistic pedestrian responses. Integrated with Monte Carlo Tree Search (MCTS), the RNN enables the robot to anticipate how

its actions influence the crowd, allowing for more socially aware planning. This approach not only maintains collision avoidance but can also shape pedestrian flow to create clearer paths. However, it relies on high-quality training data, and if the learned model does not generalize well to new social behaviors, predictions may be inaccurate. Additionally, integrating RNN-based forecasting into online planning increases computational complexity, potentially impacting real-time performance. Despite these challenges, generative models for social interaction improve state representation, making navigation safer and more efficient in dynamic environments.

Generative methods enrich robot state representation by filling in missing context enhancing planning and decision-making. LfH-based approaches generate safe training data for agile navigation, map predictors extend perception beyond sensors, and social interaction models anticipate and influence agent behavior. Domain randomization further strengthens this by procedurally generating diverse synthetic environments, exposing models to a wide range of variations in object shapes, textures, and physical properties, improving generalization to real-world scenarios. However, these generative methods risk producing inaccurate state information, leading to unsafe decisions if not managed carefully. They also add computational complexity. Procedural hallucinations and domain randomization offer safety guarantees but lack fine-grained realism, whereas data-driven generative models capture richer distributions but require extensive training data. Each method addresses different aspects of state enrichment — geometric vs. social understanding — helping robots better anticipate their surroundings while requiring careful validation.

V. OPEN PROBLEMS

Generative models have shown promise in motion planning, but several open challenges must be addressed for reliable real-world deployment. One major challenge is **generalizability**. Potential strategies are outlined in [5] and includes *behavior composition* (learning individual maneuver primitives that can be composed to tackle new tasks), *feature selection* (identifying robust state features from observations that correlate with successful outcomes), and leveraging *observation-action symmetries* (so that experience in one situation transfers to equivalent situations by symmetry). Here I will outline some personal suggestions. One approach is to *reason about the coverage* of the training data in an offline manner. This would allow one to identify where the training data is insufficient and to then augment the training data with such OOD samples [34]. Another approach is to *project OOD data to in-distribution* [20], [31]. However designing these projection functions are tricky - the projection must not ignore critical real features yet should simplify or abstract the environment enough for the learned model to handle it. Designing such projections remain an open problem.

A second challenge lies in **long-horizon planning**. Most current motion planners (learning-based or classical) are geared towards reaching a single goal or following a short

sequence of waypoints. Truly long-horizon tasks – involving dozens of intermediate decisions or sub-goals – are often tackled by hierarchical methods like Task and Motion Planning (TAMP), which break the problem into high-level task sequences and low-level motions. Generative models, however, offer a promising possibility - because some models can generate entire trajectories, they might be able to generate long horizon plans. Notably, *diffusion models* have been applied to plan trajectories that have variable planning horizons [35]. Interestingly, the planning horizon in such a model is not fixed by its architecture – the model can generate short or long trajectories as needed, and the horizon can be adjusted. The open problem is how to exploit this property for complex missions: can a single generative model reliably drive a robot through a long sequence of sub-tasks?

A third open challenge is **gathering expert demonstrations** needed to train generative planners. Broadening the training distribution by generating random configurations [22] is one possible approach but is computationally expensive. The LfH paradigm offers an innovative solution, generating *self-supervised expert data* by ‘hallucinating’ configurations during random exploration. The open problem here is the quality and breadth of these hallucinated configurations. Overly conservative hallucinations lead to suboptimal behavior while over simple hallucinations would be akin to noise. Reasoning about the coverage of the hallucinations and to proactively increase coverage while maintaining quality is an interesting research direction. Moreover, developing exploration strategies that could have the robot actively test novel maneuvers to generate novel hallucinations is another interesting research direction.

VI. CONCLUSIONS

In this survey I provide a high-level view of the use of generative methods in motion planning, focusing on their role in the motion planning pipeline, highlighting key trends. Generative models are primarily used as standalone planners, as hybrid enhancements to classical planners, or as tools for augmenting perception, with hybrid approaches—particularly those integrated with sampling-based planners—being the most prevalent. While generative models have significantly improved sampling efficiency and adaptability in navigation tasks, their application in manipulation and grasping remains limited, presenting a key area for future research. Diffusion models have recently gained traction, outperforming earlier generative methods in complex, high-dimensional planning problems, while energy-based models remain underexplored. The growing trend of combining multiple generative techniques within a single pipeline suggests that hybrid approaches may offer the best balance between efficiency and robustness. Moving forward, challenges such as improving generalization, integrating generative priors into optimization frameworks, and expanding their use in real-world robotics must be addressed. This survey provides a structured foundation for advancing generative motion planning, guiding future research toward more effective and scalable solutions.

REFERENCES

- [1] B. Ichter, J. Harrison, and M. Pavone, "Learning sampling distributions for robot motion planning," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2018, pp. 7087–7094.
- [2] J. Urain, A. T. Le, A. Lambert, G. Chalvatzaki, B. Boots, and J. Peters, "Learning implicit priors for motion optimization," in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2022, pp. 7672–7679.
- [3] J. Urain, N. Funk, J. Peters, and G. Chalvatzaki, "Se (3)-diffusionfields: Learning smooth cost functions for joint grasp and motion optimization through diffusion," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 5923–5930.
- [4] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," *The International Journal of Robotics Research*, p. 02783649241273668, 2023.
- [5] J. Urain, A. Mandlekar, Y. Du, M. Shafiuallah, D. Xu, K. Fragkiadaki, G. Chalvatzaki, and J. Peters, "Deep generative models in robotics: A survey on learning from multimodal demonstrations," *arXiv preprint arXiv:2408.04380*, 2024.
- [6] X. Xiao, B. Liu, G. Warnell, and P. Stone, "Motion planning and control for mobile robot navigation using machine learning: a survey," *Autonomous Robots*, vol. 46, no. 5, pp. 569–597, 2022.
- [7] S. M. LaValle and J. J. Kuffner Jr, "Randomized kinodynamic planning," *The international journal of robotics research*, vol. 20, no. 5, pp. 378–400, 2001.
- [8] L. E. Kavraki, P. Svestka, J.-C. Latombe, and M. H. Overmars, "Probabilistic roadmaps for path planning in high-dimensional configuration spaces," *IEEE transactions on Robotics and Automation*, vol. 12, no. 4, pp. 566–580, 1996.
- [9] M. Kalakrishnan, S. Chitta, E. Theodorou, P. Pastor, and S. Schaal, "Stomp: Stochastic trajectory optimization for motion planning," in *2011 IEEE international conference on robotics and automation*. IEEE, 2011, pp. 4569–4574.
- [10] N. Ratliff, M. Zucker, J. A. Bagnell, and S. Srinivasa, "Chomp: Gradient optimization techniques for efficient motion planning," in *2009 IEEE international conference on robotics and automation*. IEEE, 2009, pp. 489–494.
- [11] D. P. Kingma, M. Welling *et al.*, "Auto-encoding variational bayes," 2013.
- [12] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial networks," *Communications of the ACM*, vol. 63, no. 11, pp. 139–144, 2020.
- [13] D. Rezende and S. Mohamed, "Variational inference with normalizing flows," in *International conference on machine learning*. PMLR, 2015, pp. 1530–1538.
- [14] Y. LeCun, S. Chopra, R. Hadsell, M. Ranzato, F. Huang *et al.*, "A tutorial on energy-based learning," *Predicting structured data*, vol. 1, no. 0, 2006.
- [15] G. E. Hinton, "Training products of experts by minimizing contrastive divergence," *Neural computation*, vol. 14, no. 8, pp. 1771–1800, 2002.
- [16] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," *Advances in neural information processing systems*, vol. 33, pp. 6840–6851, 2020.
- [17] Y. Wang, X. Duan, and J. He, "Learning-based motion planning with mixture density networks," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 16 482–16 488.
- [18] K. Katyal, K. Popek, C. Paxton, P. Burlina, and G. D. Hager, "Uncertainty-aware occupancy map prediction using generative networks for robot navigation," in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 5453–5459.
- [19] Z. Wang, X. Xiao, A. J. Nettekoven, K. Umasankar, A. Singh, S. Bommakanti, U. Topcu, and P. Stone, "From agile ground to aerial navigation: Learning from learned hallucination," in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2021, pp. 148–153.
- [20] X. Xiao, B. Liu, G. Warnell, and P. Stone, "Toward agile maneuvers in highly constrained spaces: Learning from hallucination," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 1503–1510, 2021.
- [21] X. Xiao, B. Liu, and P. Stone, "Agile robot navigation through hallucinated learning and sober deployment," in *2021 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2021, pp. 7316–7322.
- [22] J. Tobin, L. Biewald, R. Duan, M. Andrychowicz, A. Handa, V. Kumar, B. McGrew, A. Ray, J. Schneider, P. Welinder *et al.*, "Domain randomization and generative models for robotic grasping," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 3482–3489.
- [23] J. Carvalho, A. T. Le, M. Baierl, D. Koert, and J. Peters, "Motion planning diffusion: Learning and planning of robot motions with diffusion models," in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2023, pp. 1916–1923.
- [24] C.-F. Yang, H. Xu, T.-L. Wu, X. Gao, K.-W. Chang, and F. Gao, "Planning as inpainting: A generative framework for realistic embodied path planning," in *NeurIPS 2024 Workshop on Open-World Agents*, 2024.
- [25] J. Yamada, C.-M. Hung, J. Collins, I. Havoutis, and I. Posner, "Leveraging scene embeddings for gradient-based motion planning in latent space," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 5674–5680.
- [26] C.-M. Hung, S. Zhong, W. Goodwin, O. P. Jones, M. Engelcke, I. Havoutis, and I. Posner, "Reaching through latent space: From joint statistics to path planning in manipulation," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 5334–5341, 2022.
- [27] C. Acar and K. P. Tee, "Approximating constraint manifolds using generative models for sampling-based constrained motion planning," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 8451–8457.
- [28] N. Ma, J. Wang, J. Liu, and M. Q.-H. Meng, "Conditional generative adversarial networks for optimal path planning," *IEEE Transactions on Cognitive and Developmental Systems*, vol. 14, no. 2, pp. 662–671, 2021.
- [29] L. Wang, L. Liu, and X. Lu, "Robot path planning based on generative learning particle swarm optimization," *IEEE Access*, 2024.
- [30] Z. Li, J. Wang, and M. Q.-H. Meng, "Efficient heuristic generation for robot path planning with recurrent generative model," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 7386–7392.
- [31] T. Power and D. Berenson, "Variational inference mpc using normalizing flows and out-of-distribution projection," *Robotics: Science and Systems 2022.*, 2022.
- [32] J. Morgan, D. Millard, and G. S. Sukhatme, "Cppflow: Generative inverse kinematics for efficient and robust cartesian path planning," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 12 279–12 785.
- [33] S. Eiffert, H. Kong, N. Pirmarzdashti, and S. Sukkarieh, "Path planning in dynamic environments using generative rnns and monte carlo tree search," in *2020 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2020, pp. 10 263–10 269.
- [34] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the fourteenth international conference on artificial intelligence and statistics*. JMLR Workshop and Conference Proceedings, 2011, pp. 627–635.
- [35] M. Janner, Y. Du, J. B. Tenenbaum, and S. Levine, "Planning with diffusion for flexible behavior synthesis," *arXiv preprint arXiv:2205.09991*, 2022.